

CSE445 – Machine Learning

Complete Study Guide: Support Vector Machine (SVM) — Part I & Part II

(Lectures 16 and 17, Dr. Jilan Samiuddin, North South University)

PART A: SVM — PART I (Lecture 16)

A1. Representing Samples as Vectors

Any data sample with multiple features can be represented as a **vector** in feature space.

Example from the slides: Patients described by (Cholesterol, Systolic BP, Age) are each a point/vector in 3D space:

Patient Cholesterol Systolic BP Age Vector (tail at origin)

1	150	110	35	(150, 110, 35)
2	250	120	30	(250, 120, 30)
3	140	160	65	(140, 160, 65)
4	300	180	45	(300, 180, 45)

Key convention: We assume the **tail of every vector is at the origin (0,0,0)**, so a vector can be represented simply by the coordinates of its **arrow-head (tip)** — i.e., we can treat vectors as points. This convention underlies all the vector/dot-product math used throughout SVM.

A2. Optimization — The Foundation of SVM

Definition: Optimization is the process of finding the **"best" solution** from all feasible options.

It involves three components (classic definition question):

Objective Function: the mathematical expression we want to maximize or minimize (e.g., cost, profit, error).

Decision Variables: the elements we can control/adjust (e.g., selling price, units to manufacture).

Constraints: equations/inequalities that restrict the decision variables (e.g., budget limits, minimum salary).

Convex Functions

A function is **convex** if it lies **below** the straight line segment joining any two points on it, for every pair of points in the interval.

Key property (very frequently tested): For a convex function, **any local minimum is also the global minimum**.

Diagram: A convex function looks like a single "bowl" (one minimum). A non-convex function can have multiple dips — a **local minimum** that is NOT the lowest point (global minimum) elsewhere on the curve.

Quadratic Programming (QP)

QP is a special type of optimization problem where:

The **objective function is quadratic**

Subject to **linear constraints**

Convex QP problems have convex objective functions — these are easy and efficient to solve because, thanks to convexity, **every local minimum is a global minimum**.

Example QP problem (from slides):

Consider $x = (x_1, x_2)$

Minimize $(1/2)||x||^2$ subject to $x_1 + x_2 - 1 \geq 0$

└─ quadratic objective ─┘
└─ linear constraint ─┘

Why this matters for SVM: As we'll see, the SVM training problem is formulated as exactly this kind of convex QP problem — which is precisely why SVM has a reliable, unique, globally-optimal solution (unlike the Perceptron).

A3. The Problem with Perceptron (Motivation for SVM)

Given linearly separable data, **many different lines (hyperplanes)** can separate the two classes perfectly.

Perceptron does not provide a unique solution — depending on initialization/update order, it can converge to any one of the many valid separating hyperplanes.

Perceptron's guarantee: it only guarantees convergence (in a finite number of updates) *if* the data is linearly separable — but the resulting hyperplane can have an **arbitrarily small margin** (i.e., it could pass very close to some data points).

Goal of Perceptron: Find *any* hyperplane that correctly separates the data. It does **not** explicitly try to maximize the margin.

This raises the natural question: "*Which one is the best among all possible linear decision boundaries?*"
 → **SVM's answer: the one with the maximum margin.**

A4. Support Vector Machine (SVM) — Core Idea

Case 1: Linearly Separable Data

We want to find a classifier (hyperplane) that separates negative instances from positive ones. An **infinite number** of separating hyperplanes can exist.

SVM's key idea: find the hyperplane that **maximizes the gap (margin)** between the data points that lie closest to the boundary — these closest points are called "**support vectors**."

Linear SVM (Hard Margin)

Margin = the width that the decision boundary could be increased by, in both directions, before it hits a data point in the training set.

The linear classifier with the maximum margin is considered the best one.

Why is maximum margin best? Because it is **robust to outliers** and therefore has **stronger generalization ability** — it leaves the largest possible "safe zone" around the boundary, so small perturbations in new/test data are less likely to cause misclassification.

Perceptron vs. SVM — the crucial distinction (near-guaranteed exam question):

	Perceptron	SVM
Goal	Find <i>any</i> separating hyperplane	Find the separating hyperplane with maximum margin
Margin	Not considered — can be arbitrarily small	Explicitly maximized
Solution uniqueness	Not unique	Unique (convex QP → global optimum)
Robustness/generalization	Weaker (can sit close to data points)	Stronger (largest safety buffer)

A5. Deriving the SVM Optimization Problem

Distance Between Two Parallel Hyperplanes (the key derivation)

Given two parallel hyperplanes: $w \cdot x + b_1 = 0$ and $w \cdot x + b_2 = 0$

Derivation sketch:

Let $x_2 = x_1 + t \cdot w$ (moving from a point on one hyperplane to the other, along the normal direction w)

Distance $D = ||t \cdot w|| = |t| \cdot ||w||$

Substituting into $w \cdot x_2 + b_2 = 0$ and simplifying leads to: $t = (b_1 - b_2) / ||w||^2$

Therefore: $D = (b_1 - b_2) / ||w||$

This formula is the mathematical backbone for computing the SVM margin.

Problem Formulation: Linear SVM (step by step)

The margin boundaries (the two parallel supporting hyperplanes touching the support vectors) are defined as:

$w \cdot x + b = -1$ (boundary touching negative-class support vectors)

$w \cdot x + b = +1$ (boundary touching positive-class support vectors)

The decision hyperplane itself is $w \cdot x + b = 0$, exactly in the middle.

Using the distance formula derived above (with $b_1 = b+1$, $b_2 = b-1$): $D = (b+1 - (b-1)) / ||w|| = 2 / ||w||$

Since we want to **maximize the margin $D = 2/||w||$** , this is equivalent to **minimizing $||w||$** , which (for mathematical convenience in differentiation) is written as: **minimize $(1/2)||w||^2$** (The $1/2$ is just a convenience constant that makes the derivative cleaner — it doesn't change the optimal solution.)

We also need a constraint ensuring **every data point is correctly classified**:

$w \cdot x_i + b \leq -1$ if $y_i = -1$

$w \cdot x_i + b \geq +1$ if $y_i = +1$

These two cases can be combined into **one single inequality**: $y_i(w \cdot x_i + b) \geq 1$

The Primal Formulation of Linear SVM



Objective function: minimize $(1/2)||w||^2$
 Constraint: subject to $y_i(w \cdot x_i + b) \geq 1$ for all i

This is called the **primal formulation of linear SVM**.

It is a **convex Quadratic Programming (QP) problem** — quadratic objective, linear constraints — which guarantees a unique global solution.

The Dual Formulation of Linear SVM



Objective function: maximize $\sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$
 Constraint: subject to $\alpha_i \geq 0$, and $\sum_i \alpha_i y_i = 0$

Here α_i are the **Lagrange multipliers**, one per training sample.

The weight vector is recovered from the α s: $w = \sum_i \alpha_i y_i x_i$

The prediction rule becomes: $\hat{y} = \text{sign}(\sum_i \alpha_i y_i (x_i \cdot x) + b)$

Why the dual formulation matters (very important conceptually):

The dual only depends on the data through **dot products** $\mathbf{x}_i \cdot \mathbf{x}_j$ — this is exactly the property that makes the **Kernel Trick** (Part II) possible, since we can later replace this dot product with a kernel function without ever explicitly transforming the data.

Points with $\alpha_i = 0$ are *not* support vectors (they don't affect the boundary). Points with $\alpha_i > 0$ ARE support vectors — they're the only data points that matter for defining the decision boundary.

A6. Worked Mathematical Example (Hard-Margin, Dual Formulation)

Setup:

Class +1: $\mathbf{x}_1 = (2,2)$, $\mathbf{x}_2 = (4,4)$

Class -1: $\mathbf{x}_3 = (2,4)$, $\mathbf{x}_4 = (4,2)$

Problem to solve:

$$\begin{aligned} &\text{maximize } \sum_{i=1}^4 \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \\ &\text{subject to } \sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0 \end{aligned}$$

A **Gram Matrix** is built containing all pairwise dot products $\mathbf{x}_i \cdot \mathbf{x}_j$ (e.g., $\mathbf{x}_1 \cdot \mathbf{x}_1 = 8$, $\mathbf{x}_2 \cdot \mathbf{x}_2 = 32$, $\mathbf{x}_3 \cdot \mathbf{x}_3 = 20$, $\mathbf{x}_4 \cdot \mathbf{x}_4 = 20$, and various cross terms).

Substituting these into the dual objective and simplifying with $y_1=y_2=+1$, $y_3=y_4=-1$ gives the constraint: $\alpha_1 + \alpha_2 - \alpha_3 - \alpha_4 = 0$, $\alpha_i \geq 0$.

Result (obtained via solving the QP, e.g., with `scipy.optimize.minimize`):

Optimal α : $[0.25, 0.25, 0.25, 0.25] \rightarrow$ **all four points are support vectors** (all $\alpha_i > 0$), since with only 4 points arranged symmetrically, all of them sit right on the margin boundaries.

Optimal weight vector $\mathbf{w}$: $[0.99 \approx 1, 0]$

Optimal bias b : $-2.99 \approx -3$

This produces a **vertical decision boundary at $x_1 \approx 3$** , correctly separating the left pair (Class -1: $\mathbf{x}_3=(2,4)$, $\mathbf{x}_4=(4,2)$)... note the geometry: actually here the boundary separates by the x_1 -coordinate: points with $x_1=2$ vs $x_1=4$) with maximum margin.

Python implementation notes:

The Gram matrix $K[i][j] = y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$ is precomputed.

`scipy.optimize.minimize` (method `'SLSQP'`) is used to solve the (negated, since it's a minimizer) dual objective, subject to the equality constraint $\sum \alpha_i y_i = 0$ and bounds $\alpha_i \geq 0$.

After solving: $\mathbf{w} = \sum (\alpha_i \cdot y_i) \cdot \mathbf{x}_i$, and the bias b is computed from any support vector via $b = y_k - \mathbf{w} \cdot \mathbf{x}_k$.

The final decision boundary and margins are plotted along with the highlighted support vectors.

A7. Case 2: Not Linearly Separable Data — Soft-Margin Linear SVM

Motivation: What if data is not perfectly linearly separable — e.g., due to outliers, noisy measurements, or slight non-linearity?

Approach — Slack Variables:

Assign a **slack variable** $\xi_i \geq 0$ to each instance.

ξ_i represents how far a misclassified instance has intruded into (or across) the margin — it's 0 if the point is correctly classified (outside the margin as required), and positive if the point violates the margin (or is misclassified).

ξ_i measures the "intrusion" of point i into the margin.

Soft-Margin Optimization Formulation

Primal:

$$\text{minimize } (1/2)\|w\|^2 + C \cdot \sum_i \xi_i$$

$$\text{subject to } y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

Dual formulation:

$$\text{maximize } \sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

$$\text{subject to } 0 \leq \alpha_i \leq C, \text{ and } \sum_i \alpha_i y_i = 0$$

The **only difference from hard-margin** is the extra penalty term $C \cdot \sum \xi_i$ in the primal, and the **upper bound C** on α_i in the dual (instead of just $\alpha_i \geq 0$).

C is the **regularization parameter** that controls the trade-off between maximizing the margin and minimizing classification error.

Effect of the Regularization Parameter C (important, often skipped by students — study this carefully)

	High C	Low C
Margin behavior	Narrower margin (harder margin, stricter separation)	Wider margin (softer margin, more tolerant of violations)
Misclassification penalty	Heavily penalizes misclassifications → fewer training errors, risk of overfitting	Allows more misclassifications → smoother boundary, better generalization
Trade-off priority	Prioritizes low training error over large margin	Prioritizes wide margin over perfect training accuracy

High C**Low C**

Support vectors	The narrower the margin (higher C), the fewer support vectors the decision boundary depends on	Wider margin means more points end up as support vectors
-----------------	---	---

Visual intuition (from the $C=1000$ / $C=10$ / $C=0.1$ comparison plot):

C = 1000: Very narrow margin, very few support vectors (only points right near the boundary), boundary tightly fit to the data — higher overfitting risk.

C = 10: Moderate margin, moderate number of support vectors.

C = 0.1: Wide margin, many points become support vectors, boundary is smoother/more general.

One-line memorable rule: *The bigger the C, the more penalty SVM gets for misclassifying → narrower the margin → fewer support vectors.*

PART B: SVM — PART II (Lecture 17)

B1. Case 3: Not Linearly Separable Data — The Kernel Trick

The problem: Sometimes data is **not linearly separable in the original input space**, no matter what line/hyperplane you draw (e.g., medicine dosage data where the two classes are interleaved along a 1D line — low dose → ineffective, medium dose → effective, high dose → ineffective again).

Key Idea: Map the input data into a **higher-dimensional space** where it **becomes linearly separable**.

Slide example: 1D dosage data that is not linearly separable is transformed via $\Phi(\mathbf{x}) = (\mathbf{x}, \mathbf{x}^2)$ into 2D space. In this transformed (x, x^2) space, a simple horizontal line can now separate the two classes perfectly (the parabola-shaped "Class 0" points sit above the line, and "Class 1" points cluster below it near the vertex).

Why Not Just Explicitly Transform the Data?

There is **no generalized thumb rule** for which transformation $\Phi(\mathbf{x})$ to use — a transformation that works for dataset D_1 is not guaranteed to work for dataset D_2 .

If the dataset is large, explicitly computing the mapping $\Phi(\mathbf{x})$ for every point is **computationally expensive** / time-consuming.

Solution: the Kernel Trick — avoid explicit transformation altogether.

Suppose we have 2D vectors $\mathbf{x} = [x_1, x_2]$ and $\mathbf{x}' = [x_1', x_2']$.

A typical 2nd-order polynomial mapping to a higher-dimensional space: $\Phi(\mathbf{x}) = [x_1^2, x_2^2, \sqrt{2} \cdot x_1 x_2, \sqrt{2} \cdot x_1, \sqrt{2} \cdot x_2, 1]$

This maps $\Phi: \mathbb{R}^2 \rightarrow \mathbb{R}^6$ (from 2 dimensions to 6 dimensions!). Their dot product: $\Phi(\mathbf{x}) \cdot \Phi(\mathbf{x}') = x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_2 x_1' x_2' + 2x_1 x_1' + 2x_2 x_2' + 1$

The problem: Computing $\Phi(\mathbf{x})$ explicitly for every data point (mapping to 6D) is computationally expensive.

The trick: Instead, use the **polynomial kernel function**: $K(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}' + 1)^2$

If you expand this algebraically: $(x_1 x_1' + x_2 x_2' + 1)^2 = x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2x_1 x_2 x_1' x_2' + 2x_1 x_1' + 2x_2 x_2' + 1$

This is EXACTLY the same expression as $\Phi(\mathbf{x}) \cdot \Phi(\mathbf{x}')$! — meaning we can compute the same result as doing the 6-dimensional dot product, using only a simple operation in the *original* 2D space. This is the entire magic/point of the kernel trick.

For larger datasets or complex kernels (like RBF or high-degree polynomials), explicit mapping becomes **completely infeasible**, making the kernel trick essential.

B3. Is the 2nd-Order Polynomial Mapping Unique?

No — the mapping Φ is NOT unique. It just needs to satisfy the condition: $K(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x}) \cdot \Phi(\mathbf{x}')$

Any transformation Φ that produces the same dot-product result works equally well. Examples of valid variations:

Different scaling (e.g., dropping the $\sqrt{2}$ factors and rescaling)

Different feature ordering (grouping terms differently)

Higher dimensions: for a 3D input, the 2nd-degree mapping would include terms like $x_1^2, x_2^2, x_3^2, x_1 x_2, x_1 x_3, x_2 x_3$, etc.

Restricted mappings: some applications might skip the linear terms or the constant bias term entirely.

Exam takeaway: What matters is not the specific Φ , but the **value the kernel function produces** — this is precisely why we can work with kernels directly and never need to know Φ explicitly.

B4. Kernel Trick Summary

We do **not need to know Φ explicitly** — we only need to define the **kernel function**: $K(\cdot, \cdot) : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$

Important note: Not every function $\mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ can be a valid kernel — it must satisfy the so-called **Mercer conditions** (details are out of scope for this course, but the exam may ask you to *name* this condition). If a function doesn't satisfy Mercer's condition, the underlying quadratic program may not be solvable.

The Four-Level Comparison (from Primal to Kernelized Dual — a great structure to memorize)

	Original Input Space	Feature Space via Mapping Φ
1. Primal	$\min (1/2)\ w\ ^2 \text{ s.t. } y_i(w^T x_i + b) \geq 1; \hat{y} = \text{sign}(w^T x + b)$	$\min (1/2)\ w\ ^2 \text{ s.t. } y_i(w^T \Phi(x_i) + b) \geq 1; \hat{y} = \text{sign}(w^T \Phi(x) + b)$
2. Dual	$\text{maximize } \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \text{ s.t. } \alpha_i \geq 0, \sum \alpha_i y_i = 0; \hat{y} = \text{sign}(\sum \alpha_i y_i (x_i \cdot x) + b)$	$\text{maximize } \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j)) \text{ s.t. } \alpha_i \geq 0, \sum \alpha_i y_i = 0; \hat{y} = \text{sign}(\sum \alpha_i y_i (\Phi(x_i) \cdot \Phi(x)) + b)$
3. Dual with Kernel Trick	—	$\text{maximize } \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j K(x_i, x_j) \text{ s.t. } \alpha_i \geq 0, \sum \alpha_i y_i = 0; \hat{y} = \text{sign}(\sum \alpha_i y_i K(x_i, x) + b)$

Conceptual chain to remember: Primal in input space $\rightarrow$ Primal in feature space (via Φ) $\rightarrow$ Dual (still needs $\Phi(x_i) \cdot \Phi(x_j)$) $\rightarrow$ **Dual with Kernel Trick** (replace the dot product with $K(x_i, x_j)$, never computing Φ explicitly).

B5. Popular Kernels (memorize the names and formulas)

Kernel Name	Formula
Linear kernel	$K(x_i, x_j) = x_i \cdot x_j$
Gaussian kernel	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\ ^2)$
Exponential kernel	$K(x_i, x_j) = \exp(-\gamma \ x_i - x_j\)$
Polynomial kernel	$K(x_i, x_j) = (p + x_i \cdot x_j)^q$
Hybrid kernel	$K(x_i, x_j) = (p + x_i \cdot x_j)^q \cdot \exp(-\gamma \ x_i - x_j\ ^2)$
Sigmoidal kernel	$K(x_i, x_j) = \tanh(k \cdot x_i \cdot x_j - \delta)$

B6. Kernel Trick Using Radial Basis Function (RBF) — Deep Dive

RBF Kernel (also known as the **Gaussian Kernel**) computes a similarity score between two data points based on their distance in the input space: $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$

The parameter γ (**gamma**) **controls the smoothness of the decision boundary**.

Higher $\gamma \rightarrow$ boundary is more sensitive to individual points $\rightarrow$ more complex/wiggly boundary (risk of overfitting).

Lower $\gamma \rightarrow$ smoother, more general boundary.

Why RBF is Special: Infinite-Dimensional Feature Space (very likely conceptual exam question)

If we expand the RBF kernel using the Taylor series:

First, expand the squared distance: $\|\mathbf{x} - \mathbf{x}'\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - 2(\mathbf{x} \cdot \mathbf{x}')$

So: $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x}\|^2) \cdot \exp(-\gamma \|\mathbf{x}'\|^2) \cdot \exp(2\gamma(\mathbf{x} \cdot \mathbf{x}'))$

The last term, $\exp(2\gamma(\mathbf{x} \cdot \mathbf{x}'))$, is expanded via the Taylor series of e^x : $\exp(2\gamma(\mathbf{x} \cdot \mathbf{x}')) = \sum_{n=0}^{\infty} \frac{(2\gamma(\mathbf{x} \cdot \mathbf{x}'))^n}{n!} = 1 + 2\gamma(\mathbf{x} \cdot \mathbf{x}') + \frac{(2\gamma)^2}{2!}(\mathbf{x} \cdot \mathbf{x}')^2 + \frac{(2\gamma)^3}{3!}(\mathbf{x} \cdot \mathbf{x}')^3 + \frac{(2\gamma)^4}{4!}(\mathbf{x} \cdot \mathbf{x}')^4 + \dots$

Each term in this infinite sum corresponds to a higher-order polynomial interaction between components of $\mathbf{x}$ and $\mathbf{x}'$ — meaning the RBF kernel implicitly captures polynomial features of **every degree, all the way to infinity**.

"Why This Matters" (the key takeaway to memorize verbatim in concept)

The RBF kernel **implicitly includes all polynomial kernels** (degree 1, 2, 3, ... up to infinity) within itself.

There is **no need to compute or store any of these polynomial features explicitly**.

The result is an **infinite-dimensional feature space**.

Yet, evaluation remains **fast**, because in practice you are just computing **a single exponential** ($\exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$) — not an infinite sum!

This is the single most important conceptual insight of Lecture 17 and a very likely "explain why RBF kernel is powerful" essay question.

Python Implementation (RBF SVM)



python

```

from sklearn.svm import SVC
from sklearn.datasets import make_circles

X, y = make_circles(n_samples=100, factor=0.5, noise=0.1, random_state=42) # non-linear toy data

gamma = 1.0
model = SVC(kernel='rbf', gamma=gamma)
model.fit(X, y)

# decision_function() is used to visualize the (implicit) decision boundary
# via a contour plot in the ORIGINAL 2D input space, even though the
# actual separation happens in an infinite-dimensional transformed space.

```

`make_circles` generates a toy dataset where one class forms a ring around the other — **not linearly separable** in 2D.

`SVC(kernel='rbf', gamma=...)` fits an RBF-kernel SVM.

The resulting decision boundary (plotted via `plt.contour`) is a **closed curve** (circle-like), correctly separating the inner and outer classes — something a linear SVM could never do, but an RBF-kernel SVM handles naturally by implicitly working in an infinite-dimensional space.

PART C: PROBABLE EXAM QUESTIONS & ANSWERS

C1. Brief / Definition-Type Questions

Q1. What is a support vector? A: A data point that lies exactly on (or defines) the margin boundary — the closest points to the separating hyperplane. These are the only points that determine/influence the position of the decision boundary; all other points (with $\alpha_i=0$ in the dual) have no effect.

Q2. Define margin in the context of SVM. A: The margin is the width by which the decision boundary could be increased in both directions before it touches a data point in the training set. SVM seeks the hyperplane with the maximum margin.

Q3. What is a convex function, and why is it important for SVM? A: A convex function lies below the straight line joining any two of its points. Its key property is that any local minimum is also the global minimum. This matters for SVM because the SVM optimization problem is formulated as a convex Quadratic Program, guaranteeing a unique, globally optimal solution can always be found.

Q4. What is the Kernel Trick? A: A technique that allows SVM to operate in a high-dimensional (even infinite-dimensional) transformed feature space, and find non-linear decision boundaries,

without ever explicitly computing the transformation $\Phi(x)$ — by replacing the dot product $\Phi(x_i) \cdot \Phi(x_j)$ in the dual formulation with a kernel function $K(x_i, x_j)$ computed directly in the original input space.

Q5. What is the role of the slack variable ξ_i in soft-margin SVM? A: ξ_i measures how much a data point violates/intrudes into the margin (or is misclassified). It is 0 for points correctly classified outside the margin, and positive for points that violate the margin constraint, allowing the SVM to tolerate some misclassification on non-linearly-separable data.

Q6. What does the regularization parameter C control? A: C controls the trade-off between maximizing the margin and minimizing the classification (slack) penalty. High $C \rightarrow$ narrow margin, heavily penalizes misclassification (risk of overfitting). Low $C \rightarrow$ wide margin, tolerates more misclassification (better generalization, risk of underfitting).

Q7. What does the parameter γ control in the RBF kernel? A: γ controls the smoothness/complexity of the decision boundary — higher γ makes the boundary more sensitive to individual points (more complex, risk of overfitting); lower γ produces a smoother, more general boundary.

C2. Conceptual "Explain / Why" Questions

Q8. Why does Perceptron not provide a unique solution, but SVM does? A: Perceptron's goal is only to find *any* hyperplane that separates the classes — its convergence depends on initialization and update order, so it can settle on any of the (infinitely many) valid separating hyperplanes, sometimes with a very small margin. SVM instead solves a convex Quadratic Programming problem whose objective explicitly maximizes the margin; because the problem is convex, it has one unique global optimum, always yielding the same, maximum-margin hyperplane regardless of how it's solved.

Q9. Why do we maximize the margin in SVM instead of just finding any separating hyperplane? A: A hyperplane with the maximum margin is more robust to outliers and small perturbations in new/unseen data, since it leaves the largest possible buffer zone between the classes. This gives it stronger generalization ability compared to a hyperplane that just barely separates the training data (which might misclassify new points that fall just inside a small margin).

Q10. Explain why the SVM dual formulation is important for enabling the kernel trick. A: The dual formulation's objective function depends on the training data ONLY through pairwise dot products $(x_i \cdot x_j)$, never through the raw feature vectors individually. This means we can substitute any valid kernel function $K(x_i, x_j)$ in place of the dot product — effectively working in a transformed (possibly infinite-dimensional) feature space — without ever needing to explicitly compute the transformation $\Phi(x)$. This substitution is only possible because of how the dual formulation is structured.

Q11. Why is explicit feature mapping (computing $\Phi(x)$ directly) often impractical, motivating the kernel trick? A: There's no general rule for choosing the right transformation Φ for a given dataset — one that works for one dataset may not work for another. Also, explicitly computing $\Phi(x)$ for every data point becomes computationally expensive as the dataset grows large or the target dimensionality becomes very high (or infinite, as with RBF) — so instead we compute the kernel function directly in the original space, which is mathematically equivalent to the dot product in the transformed space but far cheaper to compute.

Q12. Why does the RBF kernel correspond to an infinite-dimensional feature space? A: Expanding the RBF kernel using the Taylor series of the exponential function shows that it equals an infinite sum of terms, each corresponding to a polynomial interaction of increasing degree between the input vectors. Since the expansion never terminates, the RBF kernel implicitly represents a mapping into an infinite-dimensional feature space — yet because we only ever need to evaluate the single closed-form exponential expression $\exp(-\gamma\|x-x'\|^2)$, computation stays fast despite the infinite dimensionality.

Q13. Why is the "1/2" included in the objective function $(1/2)\|w\|^2$? A: It's purely a mathematical convenience — when you differentiate $\|w\|^2$ (as part of solving the optimization problem via calculus), the derivative produces a factor of 2, which the 1/2 cancels out neatly. It does not change the location of the optimal solution, only simplifies the derivative expressions used in solving the QP.

Q14. Is the 2nd-order polynomial feature mapping $\Phi(x)$ unique? Explain. A: No. Any transformation Φ that satisfies $K(x, x') = \Phi(x) \cdot \Phi(x')$ for the given kernel function K is valid — there can be many equivalent mappings differing in scaling, feature ordering, additional/omitted terms (like the constant bias or a $\sqrt{2}$ scaling factor), as long as their dot product reproduces the same kernel value. What matters is the value the kernel function computes, not the specific representation of Φ .

C3. Broad / Compare-and-Contrast Questions

Q15. Compare Hard-Margin and Soft-Margin Linear SVM. A: Hard-margin SVM assumes the data is perfectly linearly separable and finds the maximum-margin hyperplane with the strict constraint $y_i(w \cdot x_i + b) \geq 1$ for every point — it cannot tolerate any misclassification. Soft-margin SVM handles data that is not perfectly separable (due to noise/outliers) by introducing slack variables $\xi_i \geq 0$ that allow controlled constraint violation, with the objective $(1/2)\|w\|^2 + C \cdot \sum \xi_i$ balancing margin size against total violation, governed by the regularization parameter C .

Q16. Compare the Primal and Dual formulations of SVM. A: The Primal formulation directly optimizes over w and b : minimize $(1/2)\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$. The Dual formulation instead optimizes over the Lagrange multipliers α_i : maximize $\sum \alpha_i - (1/2) \sum \sum \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$ subject to $\alpha_i \geq 0$ and $\sum \alpha_i y_i = 0$. The dual is preferred in practice because (a) it depends on the data only through dot products, enabling the kernel trick, and (b) the number of dual variables equals the number of

training samples, which can be more tractable than the dimensionality of w in high-dimensional feature spaces.

Q17. Compare Perceptron and SVM in terms of goal, margin, and solution uniqueness. (See table in Section A4.)

Q18. Compare the Linear, Polynomial, and RBF kernels in terms of what kind of decision boundaries they can produce. A: The Linear kernel ($K = x \cdot x'$) produces a straight-line/hyperplane decision boundary — suitable only when data is linearly separable. The Polynomial kernel ($K = (p + x \cdot x')^q$) implicitly maps data into a higher (but finite) dimensional polynomial feature space, allowing curved (polynomial-shaped) decision boundaries of the given degree q . The RBF/Gaussian kernel ($K = \exp(-\gamma \|x - x'\|^2)$) implicitly maps into an infinite-dimensional feature space, allowing extremely flexible, highly non-linear (even closed, circular) decision boundaries, controlled by γ .

Q19. Trace through the four-level chain: Primal in Input Space $\rightarrow$ Primal in Feature Space $\rightarrow$ Dual $\rightarrow$ Dual with Kernel Trick. (See the table in Section B4 for the full formulas at each stage — be ready to write out all four.)

C4. Moderate / Applied Questions

Q20. Given a dataset that forms two concentric circles (one class inside a ring, another class in an outer ring), would you use a linear SVM or an RBF-kernel SVM? Why? A: An RBF-kernel SVM, because the two classes are not linearly separable in the original 2D input space — no straight line can separate an inner circle of points from an outer ring. The RBF kernel implicitly maps the data into an infinite-dimensional space where such a "circular" separation becomes an achievable (effectively curved) decision boundary in the original space, exactly as shown in the `make_circles()` example from the slides.

Q21. If you increase C to a very high value (e.g., $C=1000$) for a soft-margin SVM on noisy data, what is likely to happen and why? A: The SVM will very heavily penalize any misclassification, so it will draw a narrow-margin boundary that tries to correctly classify every noisy/outlier point in the training data. This significantly increases the risk of overfitting, since the boundary becomes overly tailored to the specific training data (including its noise), reducing the model's ability to generalize well to new, unseen data.

Q22. Two data points have $\alpha = 0$ after solving the dual SVM problem. What does this tell you about them? A: These points are NOT support vectors — they do not lie on or influence the margin boundary, and removing them from the training data would not change the resulting decision boundary at all. Only points with $\alpha_i > 0$ (support vectors) determine w and b .

Q23. You are given a small dataset and told to compute the SVM decision boundary using the polynomial kernel $K(x, x') = (x \cdot x' + 1)^2$. Explain, without doing the full explicit mapping, why this is more efficient than mapping each point into 6D space first. A: Because the kernel function

$K(x, x') = (x \cdot x' + 1)^2$ can be shown algebraically to equal $\Phi(x) \cdot \Phi(x')$ for the specific 6-dimensional mapping $\Phi(x) = [x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1]$. Instead of computing all six components of Φ for every point and then taking their dot product (which requires more operations and more memory), we can compute the same numeric result directly with one simple operation — squaring the sum $(x \cdot x' + 1)$ — entirely in the original, lower-dimensional space. This saves computation, and becomes essential for kernels (like RBF) where the implied feature space is infinite-dimensional and explicit mapping is literally impossible.

Q24. Why can't we simply keep increasing the degree of a polynomial kernel indefinitely to capture more complex patterns, and how does the RBF kernel solve this issue? A: As the polynomial degree increases, the corresponding explicit feature space grows extremely large (combinatorially), and even though the kernel trick avoids explicit computation, very high-degree polynomial kernels can become numerically unstable and prone to overfitting, and don't naturally scale to "infinite complexity." The RBF kernel effectively contains ALL possible polynomial degrees simultaneously (as shown by its Taylor expansion) within a single, numerically stable exponential formula controlled by just one parameter, γ — giving a smoothly tunable and computationally efficient way to access arbitrarily complex non-linear boundaries.

Q25. In the worked numerical example (Class +1: (2,2),(4,4); Class -1: (2,4),(4,2)), why did all four points turn out to be support vectors? A: Because of the symmetric geometric arrangement of the four points (a small symmetric square straddling the linear boundary), every point ends up sitting exactly on one of the two margin boundaries ($w \cdot x + b = +1$ or $w \cdot x + b = -1$) with no point strictly outside the margin — so every α_i came out strictly greater than 0 (all equal at 0.25), making all four points support vectors.

C5. Quick-Fire True/False Style (great for last-minute revision)

SVM's goal is to find any hyperplane that separates the classes. → **False** (finds the *maximum-margin* one)

A convex function can have a local minimum that is not the global minimum. → **False**

Perceptron explicitly maximizes the margin. → **False**

The SVM primal problem is a convex Quadratic Programming problem. → **True**

In the SVM dual formulation, only dot products of data points are needed. → **True**

Increasing C in soft-margin SVM widens the margin. → **False** (increasing C narrows the margin)

Slack variables are always negative. → **False** ($\xi_i \geq 0$)

The kernel trick requires explicitly computing the high-dimensional mapping $\Phi(x)$. → **False** (that's exactly what it avoids)

Every function $\mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ can be used as a valid kernel. → **False** (must satisfy Mercer's condition)

The RBF kernel corresponds to a finite-dimensional feature space. → **False** (infinite-dimensional)

Points with $\alpha_i = 0$ are support vectors. → **False** (only $\alpha_i > 0$ points are support vectors)

The polynomial mapping $\Phi(x)$ for a given kernel K is unique. → **False**

Final Tips for Exam Prep

Be ready to **derive** the margin formula $D = 2/\|w\|$ starting from the two parallel hyperplanes $w \cdot x + b = \pm 1$ — this derivation is a common "show your work" question.

Memorize the **primal vs dual** formulas for both hard-margin and soft-margin SVM, side by side.

Be able to **algebraically expand** $(x \cdot x' + 1)^2$ and show it matches $\Phi(x) \cdot \Phi(x')$ for the given 2nd-degree polynomial mapping — this exact derivation appeared explicitly in the slides.

Understand and be able to explain in your own words **why the RBF kernel implies an infinite-dimensional feature space** (via its Taylor series) — this is the single most conceptually rich idea in Lecture 17.

Know the **effect of C** (narrow/wide margin, over/underfitting, number of support vectors) and the **effect of γ** (boundary smoothness) — these are prime "applied scenario" question material.

Review the **four popular kernel formulas** (linear, polynomial, RBF/Gaussian, sigmoidal) — you may be asked to identify or write one from memory.